PawHome — Hibernate-Generated Writes

The INSERT, UPDATE and DELETE statements that are not in the source code · companion to *PawHome — SQL Query Reference*

Why these statements are not in the other document

The query reference lists the 166 SQL statements **written by hand** in the backend — 144 SELECT, 12 UPDATE, 6 DELETE, 4 INSERT. Reading only that document, it looks as though the application barely writes to the database. It writes constantly. The writes are simply not spelled out anywhere in the source, because **Hibernate generates them at runtime** from the entity classes.

Three separate mechanisms are involved, and they are easy to confuse:

	Does	What that means here
Flyway	DDL only	CREATE TABLE, ALTER TABLE, CREATE VIEW — the 19 migration files. Builds the shape of the database. Never writes a data row.
Hibernate / JPA	Single-row DML	When the code calls repository.save(pet) or .delete(x), Hibernate writes the INSERT / UPDATE / DELETE itself, from the entity's mapping annotations. 79 such call sites. This document reconstructs them.
JdbcTemplate + @Query	Set-based DML and all reads	The 166 hand-written statements in the query reference. The 22 writes among them are there because each changes many rows in one statement, which an ORM would turn into a loop.

So the short answer, if someone asks

"Where are your INSERT, UPDATE and DELETE queries?"
Hibernate writes the single-row ones, generated from the entity classes — 79 call sites, reconstructed in this document. The 22 hand-written ones in the query reference are the multi-row writes, done in SQL on purpose because a set-based UPDATE is one statement where an ORM loop is N.

How the statements below were reconstructed

Hibernate's rules for a basic entity are fixed, so the statement it emits can be derived from the mapping and then checked. **Table**: `@Table(name=…)`. **Columns**: `@Column(name=…)`, or the field name through Spring Boot's default camelCase→snake_case strategy, which this project does not override. **INSERT**: every mapped column except the IDENTITY id and anything marked `insertable = false`. **UPDATE**: every updatable column, keyed by the id. **DELETE**: by primary key.

These are not guesses. Every one of the 84 statements in the entity appendix was handed to MySQL with PREPARE, which parses a statement and validates every table and column name against the real schema without executing it — all 84 were accepted. A second check compared each reconstructed INSERT against information_schema to confirm no NOT NULL column without a default had been left out, which would mean the real statement is wider than the one printed. No discrepancies. Two faults were found and fixed by those checks: field names were being read from initializer values (a `boolean isRequired = true` *produced a column called* `true`*, and MySQL rejected the statement), and splitting a declaration on its "=" cut inside annotation arguments instead.*

Reading "INSERT or UPDATE"

`save()` is one method that emits either statement: Hibernate issues an INSERT when the entity's id is null and an UPDATE when it is set. Two cases settle it from the code. If the argument is plainly a new object — `Pet.builder()…build()` or `new Pet(…)` — there is no id yet, so it is an INSERT. If the argument is a variable loaded earlier in the same method (`User user = load(userId)`, then `save(user)`) the row already exists, so it is an UPDATE. Across the 79 sites that resolves **25 to INSERT, 33 to UPDATE and 8 to DELETE**; the remaining 13 show both.

Those 13 are less a limit of the analysis than cases where the code does not say. Most are entities that arrive as a lambda parameter rather than a variable — `findById(id).ifPresent(n -> { … save(n); })` *in NotificationService.markRead is an UPDATE to any reader, but the entity is never assigned to anything, so nothing in the text proves it. They are reported as both rather than guessed: the rule only moves a site out of the ambiguous column when the source is explicit, so it can miss one, but it cannot claim a wrong one.*
One more thing worth knowing about the UPDATE statements. Four entities carry `@DynamicUpdate` *— Pet, AdoptionApplication, DaycareAttendance and DaycareSubscription — and for those Hibernate includes only the columns that actually CHANGED, not every*

column. The full-width UPDATE is printed here because that is the shape derivable from the mapping; the real statement for those four is a subset of it, different on each call. They are marked in Part 2.

Part 1 — the 79 write call sites

Grouped by the file the call is written in. “Query” gives the statement Hibernate generates for that entity.

admin/AdminUserService.java — 5 writes

file name: `admin/AdminUserService.java`
api name: POST /api/v1/admin/users/{id}/approve
Query: (generated by Hibernate · User · UPDATE → table users · called from approve.save())

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

file name: `admin/AdminUserService.java`
api name: POST /api/v1/admin/users/{id}/verify-email
Query: (generated by Hibernate · User · UPDATE → table users · called from forceVerify.save())

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

file name: `admin/AdminUserService.java`
api name: POST /api/v1/admin/users/{id}/reject
Query: (generated by Hibernate · User · UPDATE → table users · called from reject.save())

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

file name: `admin/AdminUserService.java`
api name: POST /api/v1/admin/users/{id}/activate
POST /api/v1/admin/users/{id}/deactivate
Query: (generated by Hibernate · User · UPDATE → table users · called from setActive.save())

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

file name: `admin/AdminUserService.java`
api name: POST /api/v1/admin/users/{id}/shelter
Query: (generated by Hibernate · User · UPDATE → table users · called from setShelter.save())

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

admin/InviteService.java — 2 writes

file name: `admin/InviteService.java`
api name: POST /api/v1/auth/accept-invite
Query: (generated by Hibernate · User · UPDATE → table users · called from accept.save())

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

file name: `admin/InviteService.java`
api name: POST /api/v1/admin/users/invite
Query: (generated by Hibernate · User · INSERT → table users · called from invite.save())

```
INSERT INTO users
      (full_name,
       email,
       password_hash,
       phone,
       role,
       approval_status,
       email_verified,
       is_active,
       shelter_id,
       job_title,
       avatar_path,
       notify_applications,
       notify_boarding,
       notify_health,
       notify_email,
       created_at,
       updated_at,
       tokens_valid_from)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

application/ApplicationService.java — 2 writes

file name: `application/ApplicationService.java`
api name: POST /api/v1/applications
Query: (generated by Hibernate · AdoptionApplication · INSERT → table adoption_applications · called from apply.save())

```
INSERT INTO adoption_applications
      (pet_id,
       applicant_id,
       eligibility_attempt_id,
       quiz_score,
       applicant_name,
       applicant_email,
       applicant_phone,
       applicant_address,
       applicant_city,
       home_type,
       time_at_home,
       pet_experience,
       household_note,
       status,
       reviewed_by,
       reviewed_at,
       decline_reason_code,
       decline_message)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

file name: `application/ApplicationService.java`
api name: POST /api/v1/staff/applications/{id}/approve
Query: (generated by Hibernate · Adoption · INSERT → table adoptions · called from approve.save())

```
INSERT INTO adoptions
      (application_id,
       pet_id,
       adopter_id,
       shelter_id,
       adopted_on,
       fee_paid)
VALUES (?, ?, ?, ?, ?, ?)
```

audit/AuditService.java — 1 write

file name: `audit/AuditService.java`
api name: DELETE /api/v1/staff/pets/{id}
POST /api/v1/admin/shelters
POST /api/v1/staff/applications/{id}/approve
POST /api/v1/staff/applications/{id}/decline
POST /api/v1/staff/boarding/{id}/accept
POST /api/v1/staff/boarding/{id}/complete
POST /api/v1/staff/pets
Query: (generated by Hibernate · AuditLog · INSERT → table audit_log · called from record.save())

```
INSERT INTO audit_log
      (actor_id,
       action,
       entity_type,
       entity_id,
       detail)
VALUES (?, ?, ?, ?, ?)
```

auth/AuthService.java — 1 write

file name: `auth/AuthService.java`
api name: POST /api/v1/auth/register
Query: (generated by Hibernate · User · INSERT or UPDATE → table users · called from register.save())

```
INSERT INTO users
      (full_name,
       email,
       password_hash,
       phone,
       role,
       approval_status,
       email_verified,
       is_active,
       shelter_id,
       job_title,
       avatar_path,
       notify_applications,
       notify_boarding,
       notify_health,
       notify_email,
       created_at,
       updated_at,
       tokens_valid_from)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

auth/VerificationService.java — 2 writes

file name: `auth/VerificationService.java`
api name: POST /api/v1/auth/reset-password
Query: (generated by Hibernate · User · UPDATE → table users · called from resetPassword.save())

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

file name: `auth/VerificationService.java`
api name: POST /api/v1/auth/verify-email
Query: (generated by Hibernate · User · UPDATE → table users · called from verifyEmail.save())

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

billing/BillingService.java — 3 writes

file name: `billing/BillingService.java`
api name: POST /api/v1/staff/boarding/{id}/complete
Query: (generated by Hibernate · BillingSlip · INSERT → table billing_slips · called from issueForBoarding.save())

```
INSERT INTO billing_slips
      (slip_no,
       owner_id,
       shelter_id,
       boarding_request_id,
       subscription_id,
       period_from,
       period_to,
       units,
       unit_rate,
       total,
       issued_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

file name: `billing/BillingService.java`
api name: Scheduled job — MonthlyBillingJob.billTheMonth (@Scheduled)
Query: (generated by Hibernate · BillingSlip · INSERT → table billing_slips · called from issueForSubscription.save())

```
INSERT INTO billing_slips
      (slip_no,
       owner_id,
       shelter_id,
       boarding_request_id,
       subscription_id,
       period_from,
       period_to,
       units,
       unit_rate,
       total,
       issued_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

file name: `billing/BillingService.java`
api name: POST /api/v1/staff/boarding/{id}/complete
Query: (generated by Hibernate · BillingSlipItem · INSERT → table billing_slip_items · called from writeLines.save())

```
INSERT INTO billing_slip_items
      (slip_id,
       description,
       quantity,
       unit_price,
       sort_order)
VALUES (?, ?, ?, ?, ?)
```

boarding/BoardingService.java — 13 writes

file name: `boarding/BoardingService.java`
api name: POST /api/v1/staff/boarding/{id}/accept
Query: (generated by Hibernate · BoardingRequest · UPDATE → table boarding_requests · called from accept.save())

```
UPDATE boarding_requests
   SET pet_id = ?,
       owner_id = ?,
       shelter_id = ?,
       suite_id = ?,
       quiz_attempt_id = ?,
       start_date = ?,
       end_date = ?,
       picked_up_on = ?,
       status = ?,
       emergency_contact = ?,
       vet_care_authorised = ?,
       daily_rate = ?,
       estimated_cost = ?,
       reviewed_by = ?,
       reviewed_at = ?,
       decline_message = ?
 WHERE id = ?
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/staff/boarding/{id}/charges
Query: (generated by Hibernate · BoardingCharge · INSERT → table boarding_charges · called from addCharge.save())

```
INSERT INTO boarding_charges
      (boarding_id,
       description,
       quantity,
       unit_price,
       added_by)
VALUES (?, ?, ?, ?, ?)
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/boarding/{id}/cancel
Query: (generated by Hibernate · BoardingRequest · UPDATE → table boarding_requests · called from cancel.save())

```
UPDATE boarding_requests
   SET pet_id = ?,
       owner_id = ?,
       shelter_id = ?,
       suite_id = ?,
       quiz_attempt_id = ?,
       start_date = ?,
       end_date = ?,
       picked_up_on = ?,
       status = ?,
       emergency_contact = ?,
       vet_care_authorised = ?,
       daily_rate = ?,
       estimated_cost = ?,
       reviewed_by = ?,
       reviewed_at = ?,
       decline_message = ?
 WHERE id = ?
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/staff/boarding/{id}/check-in
Query: (generated by Hibernate · BoardingRequest · UPDATE → table boarding_requests · called from checkIn.save())

```
UPDATE boarding_requests
   SET pet_id = ?,
       owner_id = ?,
       shelter_id = ?,
       suite_id = ?,
       quiz_attempt_id = ?,
       start_date = ?,
       end_date = ?,
       picked_up_on = ?,
       status = ?,
       emergency_contact = ?,
       vet_care_authorised = ?,
       daily_rate = ?,
       estimated_cost = ?,
       reviewed_by = ?,
       reviewed_at = ?,
       decline_message = ?
 WHERE id = ?
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/staff/boarding/{id}/check-in
Query: **(generated by Hibernate · Pet · UPDATE → table pets · called from checkIn.save())**

```
UPDATE pets
   SET shelter_id = ?,
       name = ?,
       species = ?,
       breed = ?,
       age_value = ?,
       age_unit = ?,
       sex = ?,
       size = ?,
       weight_kg = ?,
       status = ?,
       owner_id = ?,
       description = ?,
       is_neutered = ?,
       is_microchipped = ?,
       adoption_fee = ?,
       created_by = ?
 WHERE id = ?
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/staff/boarding/{id}/complete
Query: **(generated by Hibernate · BoardingRequest · UPDATE → table boarding_requests · called from complete.save())**

```
UPDATE boarding_requests
   SET pet_id = ?,
       owner_id = ?,
       shelter_id = ?,
       suite_id = ?,
       quiz_attempt_id = ?,
       start_date = ?,
       end_date = ?,
       picked_up_on = ?,
       status = ?,
       emergency_contact = ?,
       vet_care_authorised = ?,
       daily_rate = ?,
       estimated_cost = ?,
       reviewed_by = ?,
       reviewed_at = ?,
       decline_message = ?
 WHERE id = ?
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/staff/boarding/{id}/complete
Query: **(generated by Hibernate · Pet · UPDATE → table pets · called from complete.save())**

```
UPDATE pets
   SET shelter_id = ?,
       name = ?,
       species = ?,
       breed = ?,
       age_value = ?,
       age_unit = ?,
       sex = ?,
       size = ?,
       weight_kg = ?,
       status = ?,
       owner_id = ?,
       description = ?,
       is_neutered = ?,
       is_microchipped = ?,
       adoption_fee = ?,
       created_by = ?
 WHERE id = ?
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/staff/boarding/{id}/decline
Query: (generated by Hibernate · BoardingRequest · UPDATE → table boarding_requests · called from decline.save())

```
UPDATE boarding_requests
   SET pet_id = ?,
       owner_id = ?,
       shelter_id = ?,
       suite_id = ?,
       quiz_attempt_id = ?,
       start_date = ?,
       end_date = ?,
       picked_up_on = ?,
       status = ?,
       emergency_contact = ?,
       vet_care_authorised = ?,
       daily_rate = ?,
       estimated_cost = ?,
       reviewed_by = ?,
       reviewed_at = ?,
       decline_message = ?
 WHERE id = ?
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/staff/boarding/{id}/suite
Query: (generated by Hibernate · BoardingRequest · UPDATE → table boarding_requests · called from moveSuite.save())

```
UPDATE boarding_requests
   SET pet_id = ?,
       owner_id = ?,
       shelter_id = ?,
       suite_id = ?,
       quiz_attempt_id = ?,
       start_date = ?,
       end_date = ?,
       picked_up_on = ?,
       status = ?,
       emergency_contact = ?,
       vet_care_authorised = ?,
       daily_rate = ?,
       estimated_cost = ?,
       reviewed_by = ?,
       reviewed_at = ?,
       decline_message = ?
 WHERE id = ?
```

file name: `boarding/BoardingService.java`
api name: DELETE /api/v1/staff/boarding/{id}/charges/{chargeId}
Query: (generated by Hibernate · BoardingCharge · DELETE → table boarding_charges · called from removeCharge.delete())

```
DELETE FROM boarding_charges
 WHERE id = ?
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/boarding
Query: (generated by Hibernate · BoardingInstruction · INSERT → table boarding_instructions · called from request.save())

```
INSERT INTO boarding_instructions
      (boarding_id,
       category,
       body)
VALUES (?, ?, ?)
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/boarding
Query: (generated by Hibernate · BoardingRequest · INSERT → table boarding_requests · called from request.save())

```
INSERT INTO boarding_requests
      (pet_id,
       owner_id,
       shelter_id,
       suite_id,
       quiz_attempt_id,
       start_date,
       end_date,
       picked_up_on,
       status,
       emergency_contact,
       vet_care_authorised,
       daily_rate,
       estimated_cost,
       reviewed_by,
       reviewed_at,
       decline_message)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/staff/boarding/{id}/check-in
POST /api/v1/staff/boarding/{id}/complete
POST /api/v1/staff/boarding/{id}/suite
POST /api/v1/staff/boarding/{id}/updates
Query: (generated by Hibernate · BoardingUpdate · INSERT → table boarding_updates · called from writeUpdate.save())

```
INSERT INTO boarding_updates
      (boarding_id,
       update_type,
       note,
       photo_path,
       posted_by)
VALUES (?, ?, ?, ?, ?)
```

care/CareService.java — 7 writes

file name: `care/CareService.java`
api name: POST /api/v1/staff/care/tasks/{id}/complete
Query: (generated by Hibernate · CareTask · UPDATE → table care_tasks · called from complete.save())

```
UPDATE care_tasks
   SET pet_id = ?,
       boarding_id = ?,
       task_type = ?,
       description = ?,
       due_at = ?,
       priority = ?,
       status = ?,
       assigned_to = ?,
       completed_by = ?,
       completed_at = ?,
       due_notified_at = ?
 WHERE id = ?
```

file name: `care/CareService.java`
api name: POST /api/v1/staff/care/tasks
Query: (generated by Hibernate · CareTask · INSERT → table care_tasks · called from create.save())

```
INSERT INTO care_tasks
      (pet_id,
       boarding_id,
       task_type,
       description,
       due_at,
       priority,
       status,
       assigned_to,
       completed_by,
       completed_at,
       due_notified_at)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

file name: `care/CareService.java`
api name: DELETE /api/v1/staff/care/tasks/{id}
Query: (generated by Hibernate · CareTask · DELETE → table care_tasks · called from delete.delete())

```
DELETE FROM care_tasks
 WHERE id = ?
```

file name: `care/CareService.java`
api name: POST /api/v1/staff/care/logs
Query: (generated by Hibernate · CareLog · INSERT → table care_logs · called from logCare.save())

```
INSERT INTO care_logs
      (pet_id,
       boarding_id,
       task_id,
       task_type,
       notes,
       logged_by)
VALUES (?, ?, ?, ?, ?, ?)
```

file name: `care/CareService.java`
api name: POST /api/v1/staff/care/logs
Query: (generated by Hibernate · CareTask · UPDATE → table care_tasks · called from logCare.save())

```
UPDATE care_tasks
   SET pet_id = ?,
       boarding_id = ?,
       task_type = ?,
       description = ?,
       due_at = ?,
       priority = ?,
       status = ?,
       assigned_to = ?,
       completed_by = ?,
       completed_at = ?,
       due_notified_at = ?
 WHERE id = ?
```

file name: `care/CareService.java`
api name: POST /api/v1/staff/care/tasks/{id}/skip
Query: (generated by Hibernate · CareTask · UPDATE → table care_tasks · called from skip.save())

```
UPDATE care_tasks
   SET pet_id = ?,
       boarding_id = ?,
       task_type = ?,
       description = ?,
       due_at = ?,
       priority = ?,
       status = ?,
       assigned_to = ?,
       completed_by = ?,
       completed_at = ?,
       due_notified_at = ?
 WHERE id = ?
```

file name: `care/CareService.java`
api name: PUT /api/v1/staff/care/tasks/{id}
Query: (generated by Hibernate · CareTask · UPDATE → table care_tasks · called from update.save())

```
UPDATE care_tasks
   SET pet_id = ?,
       boarding_id = ?,
       task_type = ?,
       description = ?,
       due_at = ?,
       priority = ?,
       status = ?,
       assigned_to = ?,
       completed_by = ?,
       completed_at = ?,
       due_notified_at = ?
 WHERE id = ?
```

daycare/DaycareService.java — 2 writes

file name: `daycare/DaycareService.java`
api name: POST /api/v1/staff/daycare/attendance
Query: (generated by Hibernate · DaycareAttendance · INSERT → table daycare_attendance · called from checkIn.save())

```
INSERT INTO daycare_attendance
      (subscription_id,
       slot,
       visit_date,
       suite_id,
       checked_in_by,
       checked_out_at,
       checked_out_by)
VALUES (?, ?, ?, ?, ?, ?, ?)
```

file name: `daycare/DaycareService.java`
api name: POST /api/v1/daycare/subscriptions
Query: (generated by Hibernate · DaycareSubscription · INSERT → table daycare_subscriptions · called from subscribe.save())

```
INSERT INTO daycare_subscriptions
      (pet_id,
       owner_id,
       shelter_id,
       slot,
       monthly_price,
       quiz_attempt_id,
       start_date,
       end_date,
       status)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
```

health/VaccinationService.java — 4 writes

file name: `health/VaccinationService.java`
api name: POST /api/v1/my-pets/{id}/vaccinations
POST /api/v1/staff/pets/{petId}/vaccinations
Query: (generated by Hibernate · Vaccination · INSERT → table vaccinations · called from add.save())

```
INSERT INTO vaccinations
      (pet_id,
       vaccine_type_id,
       administered_on,
       next_due_on,
       administered_by,
       batch_lot,
       status,
       certificate_path,
       notes,
       recorded_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

file name: `health/VaccinationService.java`
api name: DELETE /api/v1/my-pets/{id}/vaccinations/{vaccinationId}
DELETE /api/v1/staff/pets/{petId}/vaccinations/{id}
Query: (generated by Hibernate · Vaccination · DELETE → table vaccinations · called from delete.delete())

```
DELETE FROM vaccinations
 WHERE id = ?
```

file name: `health/VaccinationService.java`
api name: POST /api/v1/my-pets/{id}/vaccinations/{vaccinationId}/certificate
POST /api/v1/staff/pets/{petId}/vaccinations/{id}/certificate
Query: (generated by Hibernate · Vaccination · UPDATE → table vaccinations · called from setCertificate.save())

```
UPDATE vaccinations
   SET pet_id = ?,
       vaccine_type_id = ?,
       administered_on = ?,
       next_due_on = ?,
       administered_by = ?,
       batch_lot = ?,
       status = ?,
       certificate_path = ?,
       notes = ?,
       recorded_by = ?
 WHERE id = ?
```

file name: `health/VaccinationService.java`
api name: PUT /api/v1/staff/pets/{petId}/vaccinations/{id}
Query: (generated by Hibernate · Vaccination · UPDATE → table vaccinations · called from update.save())

```
UPDATE vaccinations
   SET pet_id = ?,
       vaccine_type_id = ?,
       administered_on = ?,
       next_due_on = ?,
       administered_by = ?,
       batch_lot = ?,
       status = ?,
       certificate_path = ?,
       notes = ?,
       recorded_by = ?
 WHERE id = ?
```

notification/NotificationService.java — 2 writes

file name: `notification/NotificationService.java`
api name: POST /api/v1/notifications/{id}/read
Query: (generated by Hibernate · Notification · INSERT or UPDATE → table notifications · called from markRead.save())

```
INSERT INTO notifications
      (user_id,
       type,
       title,
       body,
       link_url,
       read_at)
VALUES (?, ?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE notifications
   SET user_id = ?,
       type = ?,
       title = ?,
       body = ?,
       link_url = ?,
       read_at = ?
 WHERE id = ?
```

file name: `notification/NotificationService.java`
api name: POST /api/v1/admin/users/{id}/approve
POST /api/v1/admin/users/{id}/reject
POST /api/v1/applications
POST /api/v1/staff/applications/{id}/approve
POST /api/v1/staff/applications/{id}/decline
POST /api/v1/staff/boarding/{id}/accept
POST /api/v1/staff/boarding/{id}/check-in
POST /api/v1/staff/boarding/{id}/complete
POST /api/v1/staff/boarding/{id}/decline
POST /api/v1/staff/boarding/{id}/suite
POST /api/v1/staff/boarding/{id}/updates
Query: **(generated by Hibernate · Notification · INSERT → table notifications · called from raise.save())**

```
INSERT INTO notifications
      (user_id,
       type,
       title,
       body,
       link_url,
       read_at)
VALUES (?, ?, ?, ?, ?, ?)
```

pet/PetService.java — 12 writes

file name: `pet/PetService.java`
api name: POST /api/v1/my-pets/{id}/photos
POST /api/v1/my-pets/{id}/photos/{photoId}/cover
POST /api/v1/staff/pets/{id}/photos
POST /api/v1/staff/pets/{id}/photos/{photoId}/cover
Query: **(generated by Hibernate · PetPhoto · INSERT or UPDATE → table pet_photos · called from clearCover.save())**

```
INSERT INTO pet_photos
      (pet_id,
       file_path,
       is_cover,
       sort_order,
       uploaded_by)
VALUES (?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE pet_photos
   SET pet_id = ?,
       file_path = ?,
       is_cover = ?,
       sort_order = ?,
       uploaded_by = ?
 WHERE id = ?
```

file name: `pet/PetService.java`
api name: POST /api/v1/staff/pets
Query: (generated by Hibernate · Pet · INSERT → table pets · called from create.save())

```
INSERT INTO pets
      (shelter_id,
       name,
       species,
       breed,
       age_value,
       age_unit,
       sex,
       size,
       weight_kg,
       status,
       owner_id,
       description,
       is_neutered,
       is_microchipped,
       adoption_fee,
       created_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

file name: `pet/PetService.java`
api name: DELETE /api/v1/staff/pets/{id}
Query: (generated by Hibernate · Pet · DELETE → table pets · called from delete.delete())

```
DELETE FROM pets
 WHERE id = ?
```

file name: `pet/PetService.java`
api name: POST /api/v1/my-pets/{id}/photos/{photoId}/cover
POST /api/v1/staff/pets/{id}/photos/{photoId}/cover
Query: (generated by Hibernate · PetPhoto · UPDATE → table pet_photos · called from promoteCover.save())

```
UPDATE pet_photos
   SET pet_id = ?,
       file_path = ?,
       is_cover = ?,
       sort_order = ?,
       uploaded_by = ?
 WHERE id = ?
```

file name: `pet/PetService.java`
api name: POST /api/v1/my-pets
Query: (generated by Hibernate · Pet · INSERT → table pets · called from registerMyPet.save())

```
INSERT INTO pets
      (shelter_id,
       name,
       species,
       breed,
       age_value,
       age_unit,
       sex,
       size,
       weight_kg,
       status,
       owner_id,
       description,
       is_neutered,
       is_microchipped,
       adoption_fee,
       created_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

file name: `pet/PetService.java`
api name: DELETE /api/v1/my-pets/{id}
Query: (generated by Hibernate · Pet · DELETE → table pets · called from removeMyPet.delete())

```
DELETE FROM pets
 WHERE id = ?
```

file name: `pet/PetService.java`
api name: DELETE /api/v1/my-pets/{id}/photos/{photoId}
DELETE /api/v1/staff/pets/{id}/photos/{photoId}
Query: **(generated by Hibernate · PetPhoto · DELETE → table pet_photos · called from removePhoto.delete())**

```
DELETE FROM pet_photos
 WHERE id = ?
```

file name: `pet/PetService.java`
api name: DELETE /api/v1/my-pets/{id}/photos/{photoId}
DELETE /api/v1/staff/pets/{id}/photos/{photoId}
Query: **(generated by Hibernate · PetPhoto · INSERT or UPDATE → table pet_photos · called from removePhoto.save())**

```
INSERT INTO pet_photos
      (pet_id,
       file_path,
       is_cover,
       sort_order,
       uploaded_by)
VALUES (?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE pet_photos
   SET pet_id = ?,
       file_path = ?,
       is_cover = ?,
       sort_order = ?,
       uploaded_by = ?
 WHERE id = ?
```

file name: `pet/PetService.java`
api name: POST /api/v1/staff/pets/{id}/status
Query: **(generated by Hibernate · Pet · UPDATE → table pets · called from setStatus.save())**

```
UPDATE pets
   SET shelter_id = ?,
       name = ?,
       species = ?,
       breed = ?,
       age_value = ?,
       age_unit = ?,
       sex = ?,
       size = ?,
       weight_kg = ?,
       status = ?,
       owner_id = ?,
       description = ?,
       is_neutered = ?,
       is_microchipped = ?,
       adoption_fee = ?,
       created_by = ?
 WHERE id = ?
```

file name: `pet/PetService.java`
api name: POST /api/v1/my-pets/{id}/photos
POST /api/v1/staff/pets/{id}/photos
Query: **(generated by Hibernate · PetPhoto · INSERT → table pet_photos · called from storePhoto.save())**

```
INSERT INTO pet_photos
      (pet_id,
       file_path,
       is_cover,
       sort_order,
       uploaded_by)
VALUES (?, ?, ?, ?, ?)
```

file name: `pet/PetService.java`
api name: PUT /api/v1/staff/pets/{id}
Query: (generated by Hibernate · Pet · UPDATE → table pets · called from update.save())

```
UPDATE pets
   SET shelter_id = ?,
       name = ?,
       species = ?,
       breed = ?,
       age_value = ?,
       age_unit = ?,
       sex = ?,
       size = ?,
       weight_kg = ?,
       status = ?,
       owner_id = ?,
       description = ?,
       is_neutered = ?,
       is_microchipped = ?,
       adoption_fee = ?,
       created_by = ?
 WHERE id = ?
```

file name: `pet/PetService.java`
api name: PUT /api/v1/my-pets/{id}
Query: (generated by Hibernate · Pet · UPDATE → table pets · called from updateMyPet.save())

```
UPDATE pets
   SET shelter_id = ?,
       name = ?,
       species = ?,
       breed = ?,
       age_value = ?,
       age_unit = ?,
       sex = ?,
       size = ?,
       weight_kg = ?,
       status = ?,
       owner_id = ?,
       description = ?,
       is_neutered = ?,
       is_microchipped = ?,
       adoption_fee = ?,
       created_by = ?
 WHERE id = ?
```

quiz/QuizService.java — 7 writes

file name: `quiz/QuizService.java`
api name: POST /api/v1/admin/quiz/questions
Query: (generated by Hibernate · QuizQuestion · INSERT → table quiz_questions · called from create.save())

```
INSERT INTO quiz_questions
      (quiz_type,
       question_text,
       answer_type,
       category,
       status,
       is_required,
       display_order,
       created_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)
```

file name: `quiz/QuizService.java`
api name: POST /api/v1/admin/quiz/questions/reorder
Query: (generated by Hibernate · QuizQuestion · UPDATE → table quiz_questions · called from reorder.saveAll())

```
UPDATE quiz_questions
   SET quiz_type = ?,
       question_text = ?,
       answer_type = ?,
       category = ?,
       status = ?,
       is_required = ?,
       display_order = ?,
       created_by = ?
 WHERE id = ?
```

file name: `quiz/QuizService.java`
api name: POST /api/v1/admin/quiz/questions/{id}/status
Query: (generated by Hibernate · QuizQuestion · UPDATE → table quiz_questions · called from setStatus.save())

```
UPDATE quiz_questions
   SET quiz_type = ?,
       question_text = ?,
       answer_type = ?,
       category = ?,
       status = ?,
       is_required = ?,
       display_order = ?,
       created_by = ?
 WHERE id = ?
```

file name: `quiz/QuizService.java`
api name: PUT /api/v1/admin/quiz/questions/{id}
Query: (generated by Hibernate · QuizQuestion · UPDATE → table quiz_questions · called from update.save())

```
UPDATE quiz_questions
   SET quiz_type = ?,
       question_text = ?,
       answer_type = ?,
       category = ?,
       status = ?,
       is_required = ?,
       display_order = ?,
       created_by = ?
 WHERE id = ?
```

file name: `quiz/QuizService.java`
api name: POST /api/v1/admin/quiz/questions
PUT /api/v1/admin/quiz/questions/{id}
Query: (generated by Hibernate · QuizOption · DELETE → table quiz_options · called from writeOptions.deleteById())

```
DELETE FROM quiz_options
 WHERE id = ?
```

file name: `quiz/QuizService.java`
api name: POST /api/v1/admin/quiz/questions
PUT /api/v1/admin/quiz/questions/{id}
Query: (generated by Hibernate · QuizOption · INSERT → table quiz_options · called from writeOptions.save())

```
INSERT INTO quiz_options
      (question_id,
       label,
       weight,
       requires_text,
       display_order)
VALUES (?, ?, ?, ?, ?)
```

file name: `quiz/QuizService.java`
api name: POST /api/v1/admin/quiz/questions
PUT /api/v1/admin/quiz/questions/{id}
Query: **(generated by Hibernate · QuizOption · UPDATE → table quiz_options · called from writeOptions.save())**

```
UPDATE quiz_options
   SET question_id = ?,
       label = ?,
       weight = ?,
       requires_text = ?,
       display_order = ?
 WHERE id = ?
```

quiz/QuizTakingService.java — 2 writes

file name: `quiz/QuizTakingService.java`
api name: POST /api/v1/quiz/{quizType}/attempts
Query: **(generated by Hibernate · QuizAnswer · INSERT or UPDATE → table quiz_answers · called from submit.saveAll())**

```
INSERT INTO quiz_answers
      (attempt_id,
       question_id,
       option_id,
       answer_text,
       scale_value)
VALUES (?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE quiz_answers
   SET attempt_id = ?,
       question_id = ?,
       option_id = ?,
       answer_text = ?,
       scale_value = ?
 WHERE id = ?
```

file name: `quiz/QuizTakingService.java`
api name: POST /api/v1/quiz/{quizType}/attempts
Query: **(generated by Hibernate · QuizAttempt · INSERT → table quiz_attempts · called from submit.save())**

```
INSERT INTO quiz_attempts
      (user_id,
       quiz_type,
       score,
       max_score,
       passed)
VALUES (?, ?, ?, ?, ?)
```

report/ReportService.java — 2 writes

file name: `report/ReportService.java`
api name: DELETE /api/v1/admin/reports/{id}
Query: **(generated by Hibernate · Report · DELETE → table reports · called from delete.delete())**

```
DELETE FROM reports
 WHERE id = ?
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (generated by Hibernate · Report · INSERT → table reports · called from generate.save())

```
INSERT INTO reports
      (file_name,
       file_path,
       report_type,
       date_from,
       date_to,
       shelter_id,
       generated_by)
VALUES (?, ?, ?, ?, ?, ?, ?)
```

shelter/ShelterService.java — 6 writes

file name: `shelter/ShelterService.java`
api name: POST /api/v1/admin/shelters/{id}/suites
POST /api/v1/shelters/mine/suites
Query: (generated by Hibernate · ShelterSuite · INSERT or UPDATE → table shelter_suites · called from addSuite.save())

```
INSERT INTO shelter_suites
      (shelter_id,
       name,
       species,
       is_active,
       capacity)
VALUES (?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE shelter_suites
   SET shelter_id = ?,
       name = ?,
       species = ?,
       is_active = ?,
       capacity = ?
 WHERE id = ?
```

file name: `shelter/ShelterService.java`
api name: POST /api/v1/admin/shelters/{id}/suites/{suiteId}/deactivate
POST /api/v1/shelters/mine/suites/{suiteId}/deactivate
Query: (generated by Hibernate · ShelterSuite · INSERT or UPDATE → table shelter_suites · called from deactivateSuite.save())

```
INSERT INTO shelter_suites
      (shelter_id,
       name,
       species,
       is_active,
       capacity)
VALUES (?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE shelter_suites
   SET shelter_id = ?,
       name = ?,
       species = ?,
       is_active = ?,
       capacity = ?
 WHERE id = ?
```

file name: `shelter/ShelterService.java`
api name: POST /api/v1/admin/shelters
PUT /api/v1/admin/shelters/{id}
Query: **(generated by Hibernate · Shelter · INSERT or UPDATE → table shelters · called from save.save())**

```
INSERT INTO shelters
      (name,
       logo_path,
       email,
       phone,
       address_line,
       city,
       state,
       zip,
       capacity,
       status,
       description,
       boarding_rate_daily,
       daycare_monthly_price,
       daycare_slot_capacity)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE shelters
   SET name = ?,
       logo_path = ?,
       email = ?,
       phone = ?,
       address_line = ?,
       city = ?,
       state = ?,
       zip = ?,
       capacity = ?,
       status = ?,
       description = ?,
       boarding_rate_daily = ?,
       daycare_monthly_price = ?,
       daycare_slot_capacity = ?
 WHERE id = ?
```

file name: `shelter/ShelterService.java`
api name: POST /api/v1/admin/shelters/{id}/logo
Query: **(generated by Hibernate · Shelter · UPDATE → table shelters · called from setLogo.save())**

```
UPDATE shelters
   SET name = ?,
       logo_path = ?,
       email = ?,
       phone = ?,
       address_line = ?,
       city = ?,
       state = ?,
       zip = ?,
       capacity = ?,
       status = ?,
       description = ?,
       boarding_rate_daily = ?,
       daycare_monthly_price = ?,
       daycare_slot_capacity = ?
 WHERE id = ?
```

file name: `shelter/ShelterService.java`
api name: POST /api/v1/admin/shelters/{id}/status
Query: (generated by Hibernate · Shelter · UPDATE → table shelters · called from setStatus.save())

```
UPDATE shelters
   SET name = ?,
       logo_path = ?,
       email = ?,
       phone = ?,
       address_line = ?,
       city = ?,
       state = ?,
       zip = ?,
       capacity = ?,
       status = ?,
       description = ?,
       boarding_rate_daily = ?,
       daycare_monthly_price = ?,
       daycare_slot_capacity = ?
 WHERE id = ?
```

file name: `shelter/ShelterService.java`
api name: PUT /api/v1/admin/shelters/{id}/suites/{suiteId}
PUT /api/v1/shelters/mine/suites/{suiteId}
Query: (generated by Hibernate · ShelterSuite · INSERT or UPDATE → table shelter_suites · called from updateSuite.save())

```
INSERT INTO shelter_suites
      (shelter_id,
       name,
       species,
       is_active,
       capacity)
VALUES (?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE shelter_suites
   SET shelter_id = ?,
       name = ?,
       species = ?,
       is_active = ?,
       capacity = ?
 WHERE id = ?
```

token/TokenService.java — 2 writes

file name: `token/TokenService.java`
api name: POST /api/v1/admin/users/invite
POST /api/v1/admin/users/{id}/approve
POST /api/v1/admin/users/{id}/resend-invite
POST /api/v1/admin/users/{id}/resend-verification
POST /api/v1/auth/forgot-password
POST /api/v1/auth/register
POST /api/v1/auth/resend-verification
Query: (generated by Hibernate · UserToken · INSERT → table user_tokens · called from issue.save())

```
INSERT INTO user_tokens
      (user_id,
       token_hash,
       token_type,
       expires_at,
       used_at,
       created_at)
VALUES (?, ?, ?, ?, ?, ?)
```

file name: `token/TokenService.java`
api name: POST /api/v1/auth/accept-invite
POST /api/v1/auth/reset-password
POST /api/v1/auth/verify-email
Query: **(generated by Hibernate · UserToken · UPDATE → table user_tokens · called from redeem.save())**

```
UPDATE user_tokens
   SET user_id = ?,
       token_hash = ?,
       token_type = ?,
       expires_at = ?,
       used_at = ?
 WHERE id = ?
```

user/MeService.java — 4 writes

file name: `user/MeService.java`
api name: POST /api/v1/me/password
Query: **(generated by Hibernate · User · INSERT or UPDATE → table users · called from changePassword.save())**

```
INSERT INTO users
      (full_name,
       email,
       password_hash,
       phone,
       role,
       approval_status,
       email_verified,
       is_active,
       shelter_id,
       job_title,
       avatar_path,
       notify_applications,
       notify_boarding,
       notify_health,
       notify_email,
       created_at,
       updated_at,
       tokens_valid_from)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

file name: `user/MeService.java`
api name: POST /api/v1/me/avatar
Query: **(generated by Hibernate · User · INSERT or UPDATE → table users · called from setAvatar.save())**

```
INSERT INTO users
      (full_name,
       email,
       password_hash,
       phone,
       role,
       approval_status,
       email_verified,
       is_active,
       shelter_id,
       job_title,
       avatar_path,
       notify_applications,
       notify_boarding,
       notify_health,
       notify_email,
       created_at,
       updated_at,
       tokens_valid_from)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

file name: `user/MeService.java`
api name: PUT /api/v1/me/preferences
Query: **(generated by Hibernate · User · INSERT or UPDATE → table users · called from setPreferences.save())**

```
INSERT INTO users
      (full_name,
       email,
       password_hash,
       phone,
       role,
       approval_status,
       email_verified,
       is_active,
       shelter_id,
       job_title,
       avatar_path,
       notify_applications,
       notify_boarding,
       notify_health,
       notify_email,
       created_at,
       updated_at,
       tokens_valid_from)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

file name: `user/MeService.java`
api name: PUT /api/v1/me
Query: **(generated by Hibernate · User · INSERT or UPDATE → table users · called from updateProfile.save())**

```
INSERT INTO users
      (full_name,
       email,
       password_hash,
       phone,
       role,
       approval_status,
       email_verified,
       is_active,
       shelter_id,
       job_title,
       avatar_path,
       notify_applications,
       notify_boarding,
       notify_health,
       notify_email,
       created_at,
       updated_at,
       tokens_valid_from)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

…or, when the id is already set, the same call emits:

```
UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?
```

Part 2 — the 28 entities

Each entity's three statements in full, once, so the statements above do not have to be read twice. 26 of the 28 are written by application code; the other two are read-only in this application.

Adoption → adoptions

```
INSERT INTO adoptions
      (application_id,
       pet_id,
       adopter_id,
       shelter_id,
       adopted_on,
       fee_paid)
VALUES (?, ?, ?, ?, ?, ?)

UPDATE adoptions
   SET application_id = ?,
       pet_id = ?,
       adopter_id = ?,
       shelter_id = ?,
       adopted_on = ?,
       fee_paid = ?
 WHERE id = ?

DELETE FROM adoptions
 WHERE id = ?
```

AdoptionApplication → adoption_applications · @DynamicUpdate — the real UPDATE carries only the changed columns

```
INSERT INTO adoption_applications
      (pet_id,
       applicant_id,
       eligibility_attempt_id,
       quiz_score,
       applicant_name,
       applicant_email,
       applicant_phone,
       applicant_address,
       applicant_city,
       home_type,
       time_at_home,
       pet_experience,
       household_note,
       status,
       reviewed_by,
       reviewed_at,
       decline_reason_code,
       decline_message)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

UPDATE adoption_applications
   SET pet_id = ?,
       applicant_id = ?,
       eligibility_attempt_id = ?,
       quiz_score = ?,
       applicant_name = ?,
       applicant_email = ?,
       applicant_phone = ?,
       applicant_address = ?,
       applicant_city = ?,
       home_type = ?,
       time_at_home = ?,
       pet_experience = ?,
       household_note = ?,
       status = ?,
       reviewed_by = ?,
       reviewed_at = ?,
       decline_reason_code = ?,
       decline_message = ?
 WHERE id = ?

DELETE FROM adoption_applications
 WHERE id = ?
```

AuditLog → audit_log

```
INSERT INTO audit_log
      (actor_id,
       action,
       entity_type,
       entity_id,
       detail)
VALUES (?, ?, ?, ?, ?)

UPDATE audit_log
   SET actor_id = ?,
       action = ?,
       entity_type = ?,
       entity_id = ?,
       detail = ?
 WHERE id = ?

DELETE FROM audit_log
 WHERE id = ?
```

BillingSlip → billing_slips

```
INSERT INTO billing_slips
      (slip_no,
       owner_id,
       shelter_id,
       boarding_request_id,
       subscription_id,
       period_from,
       period_to,
       units,
       unit_rate,
       total,
       issued_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

UPDATE billing_slips
   SET slip_no = ?,
       owner_id = ?,
       shelter_id = ?,
       boarding_request_id = ?,
       subscription_id = ?,
       period_from = ?,
       period_to = ?,
       units = ?,
       unit_rate = ?,
       total = ?,
       issued_by = ?
 WHERE id = ?

DELETE FROM billing_slips
 WHERE id = ?
```

BillingSlipItem → billing_slip_items

```
INSERT INTO billing_slip_items
      (slip_id,
       description,
       quantity,
       unit_price,
       sort_order)
VALUES (?, ?, ?, ?, ?)

UPDATE billing_slip_items
   SET slip_id = ?,
       description = ?,
       quantity = ?,
       unit_price = ?,
       sort_order = ?
 WHERE id = ?

DELETE FROM billing_slip_items
 WHERE id = ?
```

BoardingCharge → boarding_charges

```
INSERT INTO boarding_charges
      (boarding_id,
       description,
       quantity,
       unit_price,
       added_by)
VALUES (?, ?, ?, ?, ?)

UPDATE boarding_charges
   SET boarding_id = ?,
       description = ?,
       quantity = ?,
       unit_price = ?,
       added_by = ?
 WHERE id = ?

DELETE FROM boarding_charges
 WHERE id = ?
```

BoardingInstruction → boarding_instructions

```
INSERT INTO boarding_instructions
      (boarding_id,
       category,
       body)
VALUES (?, ?, ?)

UPDATE boarding_instructions
   SET boarding_id = ?,
       category = ?,
       body = ?
 WHERE id = ?

DELETE FROM boarding_instructions
 WHERE id = ?
```

BoardingRequest → boarding_requests

```
INSERT INTO boarding_requests
      (pet_id,
       owner_id,
       shelter_id,
       suite_id,
       quiz_attempt_id,
       start_date,
       end_date,
       picked_up_on,
       status,
       emergency_contact,
       vet_care_authorised,
       daily_rate,
       estimated_cost,
       reviewed_by,
       reviewed_at,
       decline_message)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

UPDATE boarding_requests
   SET pet_id = ?,
       owner_id = ?,
       shelter_id = ?,
       suite_id = ?,
       quiz_attempt_id = ?,
       start_date = ?,
       end_date = ?,
       picked_up_on = ?,
       status = ?,
       emergency_contact = ?,
       vet_care_authorised = ?,
       daily_rate = ?,
       estimated_cost = ?,
       reviewed_by = ?,
       reviewed_at = ?,
       decline_message = ?
 WHERE id = ?

DELETE FROM boarding_requests
 WHERE id = ?
```

BoardingUpdate → boarding_updates

```
INSERT INTO boarding_updates
      (boarding_id,
       update_type,
       note,
       photo_path,
       posted_by)
VALUES (?, ?, ?, ?, ?)

UPDATE boarding_updates
   SET boarding_id = ?,
       update_type = ?,
       note = ?,
       photo_path = ?,
       posted_by = ?
 WHERE id = ?

DELETE FROM boarding_updates
 WHERE id = ?
```

CareLog → care_logs

```
INSERT INTO care_logs
      (pet_id,
       boarding_id,
       task_id,
       task_type,
       notes,
       logged_by)
VALUES (?, ?, ?, ?, ?, ?)

UPDATE care_logs
   SET pet_id = ?,
       boarding_id = ?,
       task_id = ?,
       task_type = ?,
       notes = ?,
       logged_by = ?
 WHERE id = ?

DELETE FROM care_logs
 WHERE id = ?
```

CareTask → care_tasks

```
INSERT INTO care_tasks
      (pet_id,
       boarding_id,
       task_type,
       description,
       due_at,
       priority,
       status,
       assigned_to,
       completed_by,
       completed_at,
       due_notified_at)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

UPDATE care_tasks
   SET pet_id = ?,
       boarding_id = ?,
       task_type = ?,
       description = ?,
       due_at = ?,
       priority = ?,
       status = ?,
       assigned_to = ?,
       completed_by = ?,
       completed_at = ?,
       due_notified_at = ?
 WHERE id = ?

DELETE FROM care_tasks
 WHERE id = ?
```

DaycareAttendance → daycare_attendance · @DynamicUpdate — the real UPDATE carries only the changed columns

```
INSERT INTO daycare_attendance
      (subscription_id,
       slot,
       visit_date,
       suite_id,
       checked_in_by,
       checked_out_at,
       checked_out_by)
VALUES (?, ?, ?, ?, ?, ?, ?)

UPDATE daycare_attendance
   SET subscription_id = ?,
       slot = ?,
       visit_date = ?,
       suite_id = ?,
       checked_in_by = ?,
       checked_out_at = ?,
       checked_out_by = ?
 WHERE id = ?

DELETE FROM daycare_attendance
 WHERE id = ?
```

DaycareSubscription → daycare_subscriptions · @DynamicUpdate — the real UPDATE carries only the changed columns

```
INSERT INTO daycare_subscriptions
      (pet_id,
       owner_id,
       shelter_id,
       slot,
       monthly_price,
       quiz_attempt_id,
       start_date,
       end_date,
       status)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)

UPDATE daycare_subscriptions
   SET pet_id = ?,
       owner_id = ?,
       shelter_id = ?,
       slot = ?,
       monthly_price = ?,
       quiz_attempt_id = ?,
       start_date = ?,
       end_date = ?,
       status = ?
 WHERE id = ?

DELETE FROM daycare_subscriptions
 WHERE id = ?
```

Notification → notifications

```
INSERT INTO notifications
      (user_id,
       type,
       title,
       body,
       link_url,
       read_at)
VALUES (?, ?, ?, ?, ?, ?)

UPDATE notifications
   SET user_id = ?,
       type = ?,
       title = ?,
       body = ?,
       link_url = ?,
       read_at = ?
 WHERE id = ?

DELETE FROM notifications
 WHERE id = ?
```

Pet → pets · @DynamicUpdate — the real UPDATE carries only the changed columns

```
INSERT INTO pets
      (shelter_id,
       name,
       species,
       breed,
       age_value,
       age_unit,
       sex,
       size,
       weight_kg,
       status,
       owner_id,
       description,
       is_neutered,
       is_microchipped,
       adoption_fee,
       created_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

UPDATE pets
   SET shelter_id = ?,
       name = ?,
       species = ?,
       breed = ?,
       age_value = ?,
       age_unit = ?,
       sex = ?,
       size = ?,
       weight_kg = ?,
       status = ?,
       owner_id = ?,
       description = ?,
       is_neutered = ?,
       is_microchipped = ?,
       adoption_fee = ?,
       created_by = ?
 WHERE id = ?

DELETE FROM pets
 WHERE id = ?
```

PetPhoto → pet_photos

```
INSERT INTO pet_photos
      (pet_id,
       file_path,
       is_cover,
       sort_order,
       uploaded_by)
VALUES (?, ?, ?, ?, ?)

UPDATE pet_photos
   SET pet_id = ?,
       file_path = ?,
       is_cover = ?,
       sort_order = ?,
       uploaded_by = ?
 WHERE id = ?

DELETE FROM pet_photos
 WHERE id = ?
```

QuizAnswer → quiz_answers

```
INSERT INTO quiz_answers
      (attempt_id,
       question_id,
       option_id,
       answer_text,
       scale_value)
VALUES (?, ?, ?, ?, ?)

UPDATE quiz_answers
   SET attempt_id = ?,
       question_id = ?,
       option_id = ?,
       answer_text = ?,
       scale_value = ?
 WHERE id = ?

DELETE FROM quiz_answers
 WHERE id = ?
```

QuizAttempt → quiz_attempts

```
INSERT INTO quiz_attempts
      (user_id,
       quiz_type,
       score,
       max_score,
       passed)
VALUES (?, ?, ?, ?, ?)

UPDATE quiz_attempts
   SET user_id = ?,
       quiz_type = ?,
       score = ?,
       max_score = ?,
       passed = ?
 WHERE id = ?

DELETE FROM quiz_attempts
 WHERE id = ?
```

QuizOption → quiz_options

```
INSERT INTO quiz_options
      (question_id,
       label,
       weight,
       requires_text,
       display_order)
VALUES (?, ?, ?, ?, ?)

UPDATE quiz_options
   SET question_id = ?,
       label = ?,
       weight = ?,
       requires_text = ?,
       display_order = ?
 WHERE id = ?

DELETE FROM quiz_options
 WHERE id = ?
```

QuizQuestion → quiz_questions

```
INSERT INTO quiz_questions
      (quiz_type,
       question_text,
       answer_type,
       category,
       status,
       is_required,
       display_order,
       created_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)

UPDATE quiz_questions
   SET quiz_type = ?,
       question_text = ?,
       answer_type = ?,
       category = ?,
       status = ?,
       is_required = ?,
       display_order = ?,
       created_by = ?
 WHERE id = ?

DELETE FROM quiz_questions
 WHERE id = ?
```

Report → reports

```
INSERT INTO reports
      (file_name,
       file_path,
       report_type,
       date_from,
       date_to,
       shelter_id,
       generated_by)
VALUES (?, ?, ?, ?, ?, ?, ?)

UPDATE reports
   SET file_name = ?,
       file_path = ?,
       report_type = ?,
       date_from = ?,
       date_to = ?,
       shelter_id = ?,
       generated_by = ?
 WHERE id = ?

DELETE FROM reports
 WHERE id = ?
```

Shelter → shelters

```
INSERT INTO shelters
      (name,
       logo_path,
       email,
       phone,
       address_line,
       city,
       state,
       zip,
       capacity,
       status,
       description,
       boarding_rate_daily,
       daycare_monthly_price,
       daycare_slot_capacity)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

UPDATE shelters
   SET name = ?,
       logo_path = ?,
       email = ?,
       phone = ?,
       address_line = ?,
       city = ?,
       state = ?,
       zip = ?,
       capacity = ?,
       status = ?,
       description = ?,
       boarding_rate_daily = ?,
       daycare_monthly_price = ?,
       daycare_slot_capacity = ?
 WHERE id = ?

DELETE FROM shelters
 WHERE id = ?
```

ShelterSuite → shelter_suites

```
INSERT INTO shelter_suites
      (shelter_id,
       name,
       species,
       is_active,
       capacity)
VALUES (?, ?, ?, ?, ?)

UPDATE shelter_suites
   SET shelter_id = ?,
       name = ?,
       species = ?,
       is_active = ?,
       capacity = ?
 WHERE id = ?

DELETE FROM shelter_suites
 WHERE id = ?
```

Trait → traits (never written by application code)

```
INSERT INTO traits
      (label,
       slug,
       is_active)
VALUES (?, ?, ?)

UPDATE traits
   SET label = ?,
       slug = ?,
       is_active = ?
 WHERE id = ?

DELETE FROM traits
 WHERE id = ?
```

User → users

```
INSERT INTO users
      (full_name,
       email,
       password_hash,
       phone,
       role,
       approval_status,
       email_verified,
       is_active,
       shelter_id,
       job_title,
       avatar_path,
       notify_applications,
       notify_boarding,
       notify_health,
       notify_email,
       created_at,
       updated_at,
       tokens_valid_from)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

UPDATE users
   SET full_name = ?,
       email = ?,
       password_hash = ?,
       phone = ?,
       role = ?,
       approval_status = ?,
       email_verified = ?,
       is_active = ?,
       shelter_id = ?,
       job_title = ?,
       avatar_path = ?,
       notify_applications = ?,
       notify_boarding = ?,
       notify_health = ?,
       notify_email = ?,
       updated_at = ?,
       tokens_valid_from = ?
 WHERE id = ?

DELETE FROM users
 WHERE id = ?
```

UserToken → user_tokens

```
INSERT INTO user_tokens
      (user_id,
       token_hash,
       token_type,
       expires_at,
       used_at,
       created_at)
VALUES (?, ?, ?, ?, ?, ?)

UPDATE user_tokens
   SET user_id = ?,
       token_hash = ?,
       token_type = ?,
       expires_at = ?,
       used_at = ?
 WHERE id = ?

DELETE FROM user_tokens
 WHERE id = ?
```

Vaccination → vaccinations

```
INSERT INTO vaccinations
      (pet_id,
       vaccine_type_id,
       administered_on,
       next_due_on,
       administered_by,
       batch_lot,
       status,
       certificate_path,
       notes,
       recorded_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

UPDATE vaccinations
   SET pet_id = ?,
       vaccine_type_id = ?,
       administered_on = ?,
       next_due_on = ?,
       administered_by = ?,
       batch_lot = ?,
       status = ?,
       certificate_path = ?,
       notes = ?,
       recorded_by = ?
 WHERE id = ?

DELETE FROM vaccinations
 WHERE id = ?
```

VaccineType → vaccine_types (never written by application code)

```
INSERT INTO vaccine_types
      (name,
       species,
       is_active)
VALUES (?, ?, ?)

UPDATE vaccine_types
   SET name = ?,
       species = ?,
       is_active = ?
 WHERE id = ?

DELETE FROM vaccine_types
 WHERE id = ?
```